

I am looking for some solutions to a react app I am working on a machine that has an m1 MacBook pro and 16gb of ram that has limited resources. I am building an app that can use computer vision to review the topology of a 3d model to

Here is the direct answer to your setup: **Ollama and LM Studio cannot run SAM (Segment Anything Model)**. Ollama and LM Studio are designed specifically for text-generating Large Language Models (LLMs) and Vision-Language Models (VLMs) like LLaVA or Florence-2 using `llama.cpp` and GGUF quantizations. SAM is an image-to-mask vision model (a Vision Transformer encoder with a prompt decoder) that outputs pixel coordinates and binary masks, not text tokens.

However, running SAM locally on an M1 Mac with 16GB RAM is actually **much faster and lighter** than running an LLM through Ollama.

1. How to Run SAM Locally on Apple Silicon (M1)

Since Apple Silicon uses Unified Memory, your GPU and CPU share that 16GB RAM. You can run **SAM 2** (Meta's faster, lighter upgraded model) natively in PyTorch using Apple's Metal Performance Shaders (`mps` acceleration).

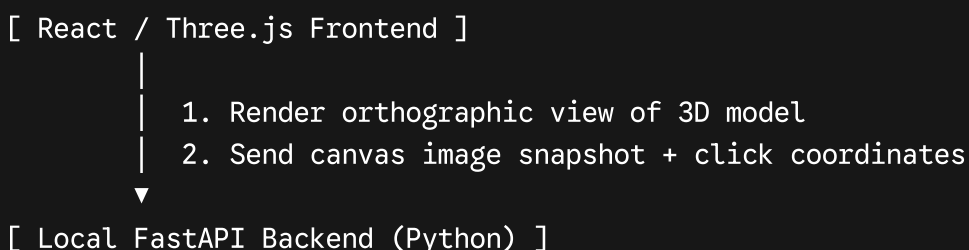
Recommended Model Weight

For a 16GB M1 Mac, use `sam2_hiera_small` or `sam2_hiera_base_plus`.

- **RAM footprint:** ~1.5 GB to 3 GB.
- **Inference time:** ~50–150ms per prompt on M1 GPU.
- **Accuracy:** High boundary precision for mechanical and organic 3D geometry outlines.

2. Architecture for Your React App

Since React runs in the browser, processing raw 3D mesh geometry with heavy AI locally inside WebAssembly can lag. The standard architecture looks like this:



3. Run SAM 2 on M1 GPU (`device="mps"`)
4. Return 2D pixel mask / polygon contour



[Mesh Cutting Engine]

5. Raycast 2D mask onto 3D mesh vertices (Trimesh / Three-bvh)
6. Execute mesh split (CSG boolean operations)

3. Python Backend Implementation (SAM 2 on M1 MPS)

Here is a minimal Python snippet using FastAPI to set up SAM 2 with Apple Metal acceleration:

Python



```
import torch
from fastapi import FastAPI, UploadFile, File, Form
from PIL import Image
import io
import numpy as np
from sam2.build_sam import build_sam2
from sam2.sam2_image_predictor import SAM2ImagePredictor

app = FastAPI()

# 1. Enable Apple Silicon GPU acceleration (MPS)
device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")

# 2. Load SAM 2 checkpoint
sam2_checkpoint = "./checkpoints/sam2_hiera_small.pt"
model_cfg = "sam2_hiera_s.yaml"

sam2_model = build_sam2(model_cfg, sam2_checkpoint, device=device)
predictor = SAM2ImagePredictor(sam2_model)

@app.post("/segment")
async def segment_mesh_view(
    file: UploadFile = File(...),
    point_x: int = Form(...),
    point_y: int = Form(...)
):
    # Load image snapshot from Three.js canvas
    image_bytes = await file.read()
    image = Image.open(io.BytesIO(image_bytes)).convert("RGB")
```

```
# Run SAM 2 inference
predictor.set_image(np.array(image))

input_point = np.array([[point_x, point_y]])
input_label = np.array([1]) # 1 = foreground point

masks, scores, _ = predictor.predict(
    point_coords=input_point,
    point_labels=input_label,
    multimask_output=False
)

# Return binary mask array or polyline contours to React
return {"mask": masks[0].tolist(), "score": float(scores[0])}
```

4. Alternate Option: Local VLM for Text Prompts

If your goal is to prompt the model with text (e.g., *"Select the multi-part joints"* or *"Highlight the split boundaries for printing"*), SAM alone won't work because it requires point clicks or bounding boxes.

If you want text-based segment selection:

1. **SAM 3 / Grounding DINO + SAM:** Use Grounding DINO to turn text into a 2D bounding box, then feed that box into SAM 2.
2. **Moondream2 or Florence-2:** Small vision-language models (~1.6B parameters) that can run via Ollama or `transformers` on M1 to return bounding box coordinates from text descriptions.

Are you planning to have the user manually click key points on the 3D model canvas to trigger the cuts, or are you looking for an automated system where AI determines the optimal seams entirely on its own?

I don't need to be able to prompt the model. I just want to upload the file [obj, stil, 3mf, ply, gltf etc and have it automatically do the segmentation. At that point I would like to be able to make edits and adjustments but my main concern is that the initial segmentation is really good.

To achieve high-accuracy, zero-prompt automatic segmentation on an M1 Mac, you actually have two powerful approaches available.

Since you don't want to manually prompt the model, you don't need to rely on prompt-based inference. You can use **SAM 2's Automatic Mask Generator** or **CoACD (Collision-Aware Approximate Convex Decomposition)**.

Approach 1: Geometry-Based Segmentation (Best for 3D Printing & Cuts)

If your primary goal is splitting models for **3D printing**, **post-print assembly**, or **painting**, pure geometry algorithms like **CoACD** often outperform vision models.

CoACD analyzes the concavity, seams, and tight joints of a 3D mesh and automatically splits it into clean, logical convex sub-parts. It runs natively in C++/Python, takes under **500MB of RAM**, and completes in seconds on an M1 Mac.

Python Backend Implementation with `coacd` & `trimesh`:

Python



```
import trimesh
import coacd
from fastapi import FastAPI, UploadFile, File
import tempfile

app = FastAPI()

@app.post("/auto-segment-geometry")
async def segment_mesh(file: UploadFile = File(...)):
    # 1. Save uploaded file to temp directory
    with tempfile.NamedTemporaryFile(suffix=".obj", delete=False) as tmp:
        tmp.write(await file.read())
        tmp_path = tmp.name

    # 2. Load mesh using Trimesh
    mesh = trimesh.load(tmp_path, force="mesh")

    # 3. Convert to CoACD format
    coacd_mesh = coacd.Mesh(mesh.vertices, mesh.faces)

    # 4. Run automatic convex decomposition
    # threshold controls how aggressively it splits concavities (0.01 to 0.1)
    parts = coacd.run(coacd_mesh, threshold=0.05)

    # 5. Extract segmented meshes to return to React
    segmented_data = []
    for idx, (verts, faces) in enumerate(parts):
        segmented_data.append(ξ
```

```

        "segment_id": idx,
        "vertices": verts.tolist(),
        "faces": faces.tolist()
    })

    return {"status": "success", "total_segments": len(parts), "segments": segmented_

```

Approach 2: SAM 2 Automatic Mask Generator (Best for Visual Parts)

If the model is a single continuous mesh without sharp geometric seams (e.g., an organic sculpture where you want to separate clothes from skin or hair), SAM 2 has a built-in

`SAM2AutomaticMaskGenerator`.

Instead of waiting for a prompt, it overlay-grids hundreds of points across rendered orthographic views, generates all valid mask proposals, filters out duplicates, and projects those 2D masks back onto the 3D mesh vertices.

Python



```

import torch
from sam2.build_sam import build_sam2
from sam2.automatic_mask_generator import SAM2AutomaticMaskGenerator
import numpy as np

# Load SAM 2 on Apple Silicon MPS
device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
sam2_checkpoint = "./checkpoints/sam2_hiera_small.pt"
model_cfg = "sam2_hiera_s.yaml"

sam2_model = build_sam2(model_cfg, sam2_checkpoint, device=device)

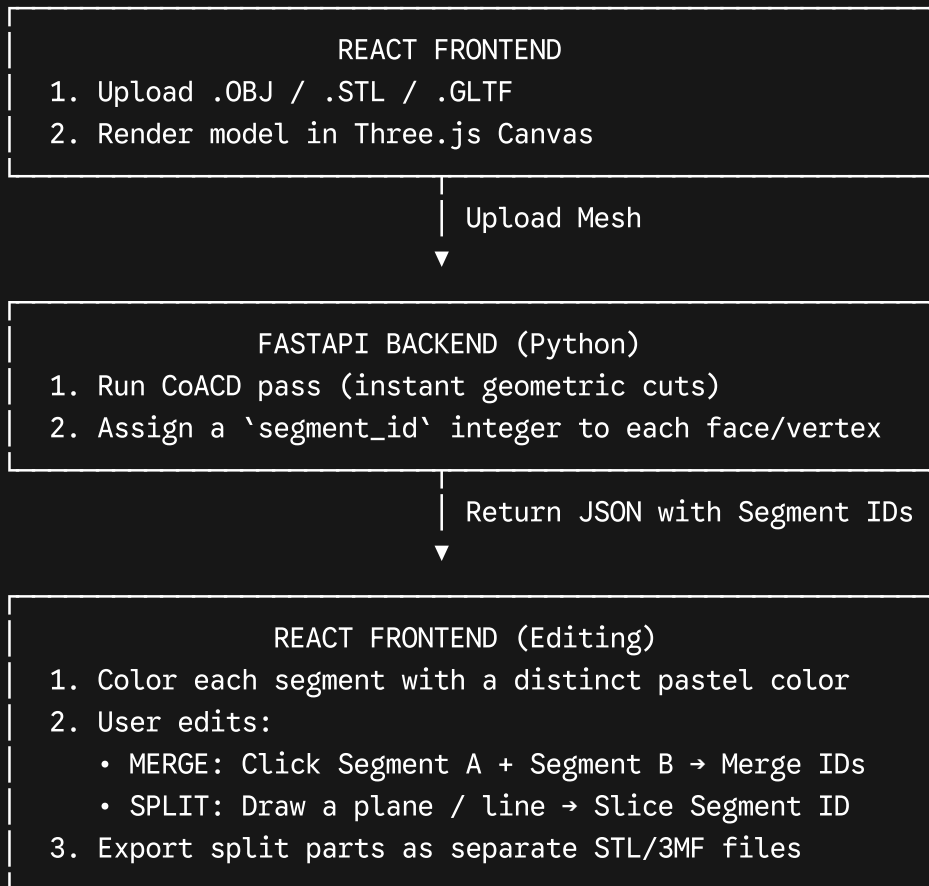
# Automatic Mask Generator evaluates the entire image automatically
mask_generator = SAM2AutomaticMaskGenerator(
    model=sam2_model,
    points_per_side=32,          # Grid density
    pred_iou_thresh=0.88,        # High confidence threshold
    stability_score_thresh=0.95
)

def auto_segment_image(image_array: np.ndarray):
    # Generates a list of all detected segments without user prompts
    masks = mask_generator.generate(image_array)
    return masks

```

3. Recommended Workflow & Interactive Edits in React

To handle both the initial automatic pass and your requirement for **manual edits/adjustments**, structure your React frontend (using Three.js / `@react-three/fiber`) like this:



Why this hybrid pipeline works best on M1:

1. **Low Resource Footprint:** CoACD takes < 500MB RAM, while SAM 2 takes ~2GB RAM. Your 16GB M1 Mac will comfortably handle this without thrashing memory.
2. **Deterministic Mesh Topology:** Pure geometry algorithms (CoACD) preserve manifold boundaries, making the resulting mesh parts immediately ready for 3D printing slicers (PrusaSlicer, Bambu Studio, Cura) without non-manifold edge errors.

Which type of 3D models will your users mostly be uploading—organic shapes like characters/figurines, or functional/mechanical CAD parts?